Algebraic relation oracles for the Curve25519 family: a search with

counted costs (k = 20, 2026-11-07) **Class.** Counted measurements of decision procedures in exact Python on the k = 20 miniature, combined with the parity model of `curve25519_rho_parity_bound_20261104/scripts/screen.py`. No ecbench run, no leaderboard or `benchmarks/` row. Every number is labelled measured (with its file) or derived. Every result here is an unverified assertion; no pcc claim was filed. Protocol: PROTOCOL.md (amendments at the end). Its pre-data freeze cannot be checked, so the predictions are not preregistered (see Deviations).

Question. ALGEBRAIC_SCREEN.md left a 265x gap to rho-neg at w = 0.1 for its best row, an m = 2 full-coverage base with one root-find per base point. Parity there needs about 14 native ops per trial at w = 0.1 and about 3 at w = 1. Is there a relation oracle, meaning a way to find index-calculus relations without collisions, that closes this gap?

Answer. No design found here closes it. The best non-generic oracle is 821x behind rho-neg at w = 0.1, and, in the model extrapolation only, non-generic gaps grow with r (derived, not measured: only k = 20 was run) [pcc:unverified]. The screen's 265x row turned out to be generic. Its per-base-point root-find is exactly a group addition: the two roots in x2 of S3(x1, x2, xR) are x(R + P1) and x(R − P1), and this held on 2,000 of 2,000 instances (`data/d1_m2.json`) [pcc:unverified]. So that row is baby-step giant-step with an unknown base. Charged as group operations, it is 251x behind at w = 0.1 [pcc:unverified]. The only designs within 10x at any weight are generic (BSGS-shaped). The collision floor bounds them, and the existing wwalk incumbent (about 1.8x) beats them [pcc:unverified]. Under the frozen rule, no ecbench test design is triggered.

Method

- Curve: $y^2 = x^3 + 926x^2 + x$ over F_p, p = 1,048,573, #E = 8r with r = 131,203. `fp.self_check` checks #E·P = O, rP = O on E[r], and a point of order 8 (cyclic 2-part).
- Counted unit: one F_p multiplication or squaring = 1 native op (`scripts/fp.py`). Inversion is counted through its Fermat chain (37 ops). Additions are not counted.
- Model: S(w) = [G_setup + w·N_setup + T·(1 + g + w·c) + w·N_LA] / √r, with T = (F + 1)·r/C. The rho reference is rho-neg S = 1.600 / 1.716 / 2.764 at w = 0 / 0.1 / 1, as in the screen. Gap = S ÷ S(rho-neg).
- Charges the screen leaves out are included here: base decoding (25 native and 3 group ops per base point, derived) and linear algebra ((m
  1. 1)·F² native, derived). The table also gives the setup-free gap.
- Two pricings for an oracle that computes group elements:
  - **P_G (primary):** each element is a group operation (1 GAE), the same charge as a rho step.
  - **P_N (the screen's convention):** its field multiplications are native, priced at w. At w = 0 this makes a BSGS table free, so it is not a fair comparison.
- Coverage C: exhaustive enumeration of ±P1 ± P2 (or ±P1 ± P2 ± P3) over the base. Torsion compatibility is enforced for bases outside E[r]. Hit rates on random R ∈ E[r] agree with C/r within sampling error in every measured row (`data/d1_m2.json`, `d2`, `d3`, `d5`). The D3 rows are the exception: their C/r column is the sampled hit rate of 300 trials, not enumerated coverage.

Design table (gaps to rho-neg; k = 20)

C/r = measured (enumerated) coverage, except the two D3 rows, where it is a 300-trial sampled hit rate; g = group ops per trial beyond the trial point; c = native ops per trial; exp = growth exponent of S in r, derived from the model, not measured (see the note after the table). Source files are under `data/`. Rows come from `data/table.json` (`scripts/model.py`).

id	design	class	pricing	F	C/r	trials	g	c	gap w=0	gap w=0.1	gap w=1	setup-free w=0.1	exp
screen	m=2, F=256, root-find, c = 25F (derived, screen)	generic	P_N	256	1.0	257	0	6,400	0.44	265	$1.6\cdot10^3$	265	0.5
D1	m=2, F=256, quadratic + sqrt per base point (measured 82 / base point)	generic	P_N	256	0.632	406	0	$2.1\cdot10^4$	2.0	$1.4\cdot10^3$	$8.7\cdot10^3$	$1.4\cdot10^3$	0.5
D1	m=2, shared-inversion x(R ± P1) (measured 7.1–7.3 /	generic	P_N	128	0.221	584	0	930	1.7	97	596	88	0.5

	base point), F=128												
D1	same	generic	P_G	128	0.221	584	256	0	260	251	203	242	0.5
D2	coset g·μ_d, d=1022, norm/ resultant + gcd in F_p[X]/ (X^d − c)	algebraic	P_G	495	0.611	812	0	$3.3 \cdot 10^{6}$	1.4	$4.3 \cdot 10^{5}$	$2.7 \cdot 10^{6}$	$4.3 \cdot 10^{5}$	1.35
D2	same, d=2394	algebraic	P_G	1208	0.996	1,212	0	$1.46 \cdot 10^{7}$	2.1	$2.9 \cdot 10^{6}$	$1.8 \cdot 10^{7}$	$2.9 \cdot 10^{6}$	1.35
D2	coset d=2394, per-point additions	generic	P_G	1208	0.996	1,212	2,418	0	$5.1 \cdot 10^{3}$	$5.4 \cdot 10^{3}$	$7.3 \cdot 10^{3}$	$4.7 \cdot 10^{3}$	0.5
D3	E[8]-closed base, 384 unknowns, one representative per orbit	generic	P_G	384	0.913	422	768	0	561	596	776	522	0.5
D3	same, every orbit point tested (7.74–7.91x the cost across F)	generic	P_G	384	0.897	429	6,144	0	$4.6 \cdot 10^{3}$	$4.3 \cdot 10^{3}$	$3.1 \cdot 10^{3}$	$4.2 \cdot 10^{3}$	0.5
D4	pair-sum table, hash lookup (best F=29)	generic, BSGS-shaped	P_G	29	0.0127	2,362	0	0	5.7	**5.9**	6.6	3.8	0.16
D4	eliminant Φ of the pair-sum set, batched multipoint evaluation (best F=16)	algebraic decision, generic setup	P_G	16	0.0039	4,364	0	1,150	9.0	**821**	$5.1 \cdot 10^{3}$	816	0.5
D5	m=3, direct pair loop, F=56	generic	P_G	56	0.84	68	6,496	0	761	712	454	709	0.5
D5	m=3, meet in the middle (pair table), F=56	generic, BSGS-shaped	P_N	56	0.84	68	0	426	0.41	10.9	65	4.8	0.17
D5	same	generic, BSGS-shaped	P_G	56	0.84	68	112	0	19	20	25	12	0.17
D5	m=3, F=46, F4 on S4 (T3 measured cost at F=8, a lower bound)	algebraic	P_G	46	0.633	74	0	$4.0 \cdot 10^{7}$	0.13	$4.8 \cdot 10^{5}$	$3.0 \cdot 10^{6}$	$4.8 \cdot 10^{5}$	0.53
D5	same, I2 linear 2-torsion quotient, F4 cost ÷ 100 (derived, optimistic)	algebraic	P_G	46	0.633	74	0	$4.0 \cdot 10^{5}$	0.13	$4.8 \cdot 10^{3}$	$3.0 \cdot 10^{4}$	$4.8 \cdot 10^{3}$	0.53
D7	small abscissa x ≤ 32, LLL oracle (measured: 0 of 40	algebraic	P_G	15	0 found	∞	0	$1.5 \cdot 10^{4}$	∞	∞	∞	∞	—

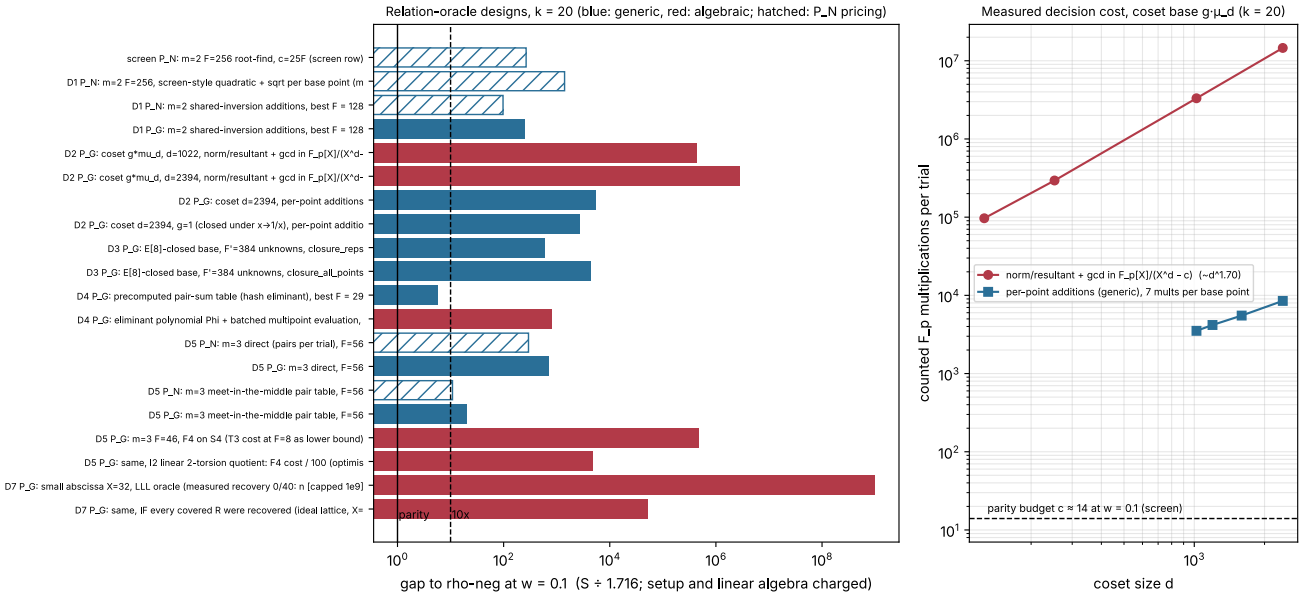
	planted relations recovered)												
D7	same, if every covered R were recovered (ideal lattice)	algebraic	P_G	15	$7.5 \cdot 10^{-4}$	$2.1 \cdot 10^{4}$	0	$1.5 \cdot 10^{4}$	37	$5.3 \cdot 10^{4}$	$3.3 \cdot 10^{5}$	$5.3 \cdot 10^{4}$	0.4

All table values are model outputs from measured counts (D1–D3, D5 direct and MITM, D7) or from derived counts, labelled in the row [pcc:unverified].

Weight-0 accounting. The D5 F4 rows and the D2 polynomial rows charge no base-decoding setup (Gs = 0), while other rows charge 3F group operations. Their w = 0 gaps (0.13 for D5 F4; 1.4 and 2.1 for D2) and the screen row's 0.44 come from native work being free at w = 0. They are weight-0 accounting artifacts, not results, and no gain is claimed from them [pcc:unverified]. The w = 0.1 and w = 1 columns of those rows are unaffected in order of magnitude. The setup was not re-charged in this pass (table values unchanged).

Exponents (exp column) are model-derived and are not measurements. Only k = 20 was run.

- Generic rows: 0.5 is a typed-in constant in `scripts/model.py` (assumption: T ∝ r/F with F ∝ √r).
- D2 polynomial rows, 1.35 = 0.5·e_poly + 0.5, where e_poly = 1.70 is fitted at k = 20, under the assumptions F ~ √r and d ~ F.
- D7 0.4 and D5 F4 0.53 are typed-in constants in `scripts/model.py` (the latter from the modelled F4 cost r^1.03 in the earlier screen).
- Only the D4 exponents (0.16 hash, 0.50 polynomial) come from re-solving the model at other r (`data/table.json`, `growth`). They are still model outputs, with no run at another r.



Sources: data/table.json (scripts/model.py), data/d2_coset.json (scripts/d2_coset.py). Model values from measured counts [pcc:unverified].

Findings per candidate

(D1) The screen's best row is generic. The roots in x2 of S3(x1, x2, xR) are x(R ± P1) on 2,000 of 2,000 instances, so a "root-find per base point" is a group addition (`data/d1_m2.json`) [pcc:unverified]. The screen-style solve (quadratic, Atkin square root, inversion) costs a measured 82 multiplications per base point, not 25 [pcc:unverified]. Sharing the denominator xR − x1 between both signs and using batch inversion brings this to 7.1–7.3 multiplications per base point [pcc:unverified]. Coverage at F = 256 is 0.632·r, not r, because pair sums collide; the measured C follows r·(1 − e^(−2F²/r)) within 1 % [pcc:unverified]. Under P_G the best measured F (128, the smallest in the sweep) is 251x behind at w = 0.1 [pcc:unverified]. Under P_N it is 97x at w = 0.1 and 596x at w = 1 [pcc:unverified].

(a / D2) A coset base g·μ_d gives no sublinear decision. The norm-resultant decision N(X) = Res_Y(S3(X, Y, xR), Y^d − c) uses only O(log d) ring operations. Each operation is a degree-d polynomial product, and the closing gcd is quadratic. The counted cost of this implementation (Karatsuba products plus a quadratic Euclidean gcd) is $9.7 \cdot 10^{4}$, $2.9 \cdot 10^{5}$, $3.3 \cdot 10^{6}$ and $1.46 \cdot 10^{7}$ multiplications at d = 126, 252, 1022 and 2394, which scales as d^1.70 (`data/d2_coset.json`) [pcc:unverified]. At d = 2394 that is 1,717x the per-point decision (8,497) for this implementation [pcc:unverified]. This is not a lower bound: a half-gcd (quasi-linear) implementation would cut the ratio to tens, not thousands (validator estimate, not run here). The conclusion that D2 is

far beyond 10x still holds at that ratio. Its yes/no answer agreed with the per-point oracle on 60 of 60 checks, 30 of them planted relations [pcc:unverified]. Coverage is real: d = 2394 with g = 5 covers 0.996·r with 1,208 unknowns [pcc:unverified]. The structure does not cut the cost. Any polynomial approach handles elements of size d, so it is Ω(d) per trial (the only general statement), and this implementation measures about d^1.7 [pcc:unverified].

- The g = 1 coset is closed under x ⟶ 1/x, which is translation by the 2-torsion point. Testing both points of each orbit doubles the per-unknown cost to 14.1 [pcc:unverified].

(b / D3) Linear 2-torsion (Edwards) does not cut the decision. At equal unknowns (128, 256, 384), the E[8]-closed base with one representative per orbit has the same counted cost as the E[r]-only base (930, 1,826 and 2,722), by construction: the `closure_reps` arm runs the same per-representative code as the E[r]-only arm, so the cost equality is tautological [pcc:unverified]. Only the hit-rate equality is an empirical comparison: the same within 4.5 % (`data/d3_torsion.json`, 300-trial sampled hit rates) [pcc:unverified]. Testing every orbit point costs 7.74–7.91x more across F for the same coverage [pcc:unverified]. The quotient merges base points that share one unknown. It removes no work per unknown. Its only lever is the F4 degree (record I2), and even an optimistic 100x F4 saving leaves $4.8 \cdot 10^3$x [pcc:unverified].

(c / D4) The precomputed eliminant is either a hash table or worse. At full coverage the covered set is all of x(E[r]). Φ is then the r-division polynomial, which a ladder decides in O(log r), but it never names the pair, so the search cost is unchanged (derived).

- At partial coverage, a hash table of the pair-sum set is BSGS-shaped: 5.9x at w = 0.1 at F = 29, growth exponent 0.16 (`data/table.json`) [pcc:unverified].
- Swapping the hash for a polynomial Φ with batched multipoint evaluation costs 821x at w = 0.1. It is dominated by the hash table it replaces [pcc:unverified].
- Its Karatsuba count model matches the counted implementation exactly at n = 30, 100 and 257 (`scripts/model.py`) [pcc:unverified].

(d / D5) m = 3 at full coverage. Coverage at F = 46 is 0.633·r, not ≥ r as the screen assumed. F = 56 reaches 0.84·r (`data/d5_m3.json`) [pcc:unverified].

- The direct pair loop costs $2.7 \cdot 10^4$ native ops per trial [pcc:unverified].
- A meet-in-the-middle pair table cuts this to 426 per trial: 2F additions per trial against a setup of F(F + 1) [pcc:unverified]. That design is generic and is 20x behind under P_G [pcc:unverified]. Under P_N it is 10.9x at w = 0.1 and 65x at w = 1 [pcc:unverified].
- Over the F sweep, the best P_N value is 7.1x at F = 32, w = 0.1, and the P_G values stay at 14.7x or more (`data/d6_tradeoff.json`) [pcc:unverified].
- Algebraic m = 3 (F4 on S4) costs at least $4.0 \cdot 10^7$ per trial (T3), which is $4.8 \cdot 10^5$x behind [pcc:unverified].

(e / D6) Coverage against decision cost. For per-point oracles, total native work is about trials × c ≈ (r/2F)·7F, roughly 3.5·r, whatever F is (derived; `data/d6_tradeoff.json`). So P_N gaps fall slowly as F shrinks, and P_G gaps stay at 250x or more [pcc:unverified]. Trading coverage for decision cost moves work between the two factors and leaves the product near r.

(own / D7) Small-abscissa base with a lattice oracle. S3 is linear in the six monomials (1, s, t, s^2, st, t^2) of s = x1 + x2 and t = x1·x2. A 6-dimensional LLL per trial costs O(1) independent of F: $1.5–2.1 \cdot 10^4$ arithmetic ops, multi-precision, so this undercounts [pcc:unverified].

- Positive control: planted vectors with entries in [−3, 3] were recovered 20 of 20 times [pcc:unverified].
- Real instances: 0 of 40 recovered at every X from 3 to 32 (`data/d7_lattice.json`) [pcc:unverified]. The Gaussian heuristic gives X_max ≈ 1.8 (derived), below the smallest base abscissa on the curve (x = 3).
- Even an ideal lattice at X = 32 is 37x behind at w = 0 [pcc:unverified]. Coverage scales as $X^2 \approx p^{(1/5)}$ at the heuristic bound (derived).

Predictions (not preregistered): outcomes

These predictions come from a PROTOCOL.md whose pre-data freeze cannot be checked (see Deviations). "Held" means consistent with the data, not a held preregistered prediction.

id	outcome
P1	held: 0 / 2,000 failures (after fixing a script bug in S3; amendment 1) [pcc:unverified]
P2	held: c = 1,826 at F = 256 [pcc:unverified]
P3	held for this implementation: 1,717x at d = 2394, cost ∝ d^1.70 (not a lower bound; half-gcd would give tens) [pcc:unverified]
P4	cost equality holds by construction; only the hit rate (within 4.5 %, sampled) was tested; testing all orbit points is 7.74–7.91x worse [pcc:unverified]
P5	held: polynomial Φ 821x; hash version generic [pcc:unverified]
P6	held: MITM P_G 20x at F = 56, against rho-neg; the collision floor for F = 56 is 8.5x of model rho (different reference) [pcc:unverified]

P7	held: best non-generic optimum 821x at w = 0.1 [pcc:unverified]
P8	**failed** in its first half: LLL recovered nothing for any X (predicted success at X ≤ 4); the second half (≥ 30x at w = 0) held [pcc:unverified]
P9	not tested: no measurement at any r other than k = 20. Its outcome is derived from model exponents (0.4 to 1.35), several of them typed-in constants, so it is a model extrapolation and not a held prediction [pcc:unverified]

Conclusion and test design

- No non-generic oracle comes within 10x of rho-neg at w = 0.1, so no single-target ecbench test design is written [pcc:unverified].
- Two generic designs come within 10x at some weight under some pricing: the pair-sum hash table (D4, 5.9x against rho-neg) and the m = 3 MITM under P_N (D5, 7.1–10.9x) [pcc:unverified].
  - Under the frozen classification they are BSGS relabelled with an unknown base.
  - Same reference for both: against model rho, the F = 29 collision floor is 6.15x and the D4 design is 10.3x (derived). The 5.9x above is against measured rho-neg and the 6.15x floor is against model rho, so they must not be compared side by side. On the same reference D4 is above the floor.
  - The lab's generic incumbent, wwalk (F = 2), already measures about 1.8x, so an ecbench test of either would be uninformative.
- What would be needed is unchanged: a relation-finding procedure whose per-trial work is below about 14 native ops at w = 0.1 while it never forms R ± P_i.
  - Every explicit-base route measured here either computes those group elements (generic) or handles a degree-d object (Ω(d)).
  - The lattice route, the only O(1)-per-trial route, has no usable coverage [pcc:unverified].

Limits

- Python counts, not ecbench counters. Additions in F_p are free. LLL ops are multi-precision.
- Linear algebra is charged as $(m + 1)F^2$ (derived). For m = 2 a spanning-tree solve costs O(F), which changes no conclusion.
- Using cycles in the m = 2 relation graph needs fewer rows, but cycles are collisions, which the generic floor covers.
- The D1 optimum is at the edge of the sweep (F = 128). Smaller F tends to about 74x under P_N at w = 0.1 (derived from the 3.5·r work law).

Deviations

- **Freeze not verifiable.** PROTOCOL.md claims to be frozen before any data, but its mtime (20:00:55) is later than every data file (19:49 to 20:00), the directory is untracked in git, and there is no hash, commit or run log of a pre-data version. The predictions are therefore treated as not preregistered. Any "held" below means consistent with the data only. Future protocols should be bound by hash or commit before the first run.
- The protocol's amendments are recorded in PROTOCOL.md, which this validation pass did not edit.
- Setup was not re-charged on the D5 F4 and D2 polynomial rows (weight-0 accounting note above).

Validation

Source: `validator-report.json` (status pass_with_errata). Independent code reproduced the headline numbers. Every result remains an unverified assertion, with no pcc claim. Errata and how each was applied to this README:

1. **P9 / exponents are model-derived (refuted as measured).** Relabelled as derived. The answer sentence, exp column header, P9 row and an assumptions note name which exponents are typed-in constants (0.5, 0.4, 0.53), which rest on assumptions (D2 1.35) and which re-solve the model (D4).
2. **Protocol freeze unverifiable.** Disclosed in Deviations. Predictions retitled "not preregistered".
3. **D2 d^1.70 and 1,717x.** Relabelled as the cost of this implementation (Karatsuba plus quadratic gcd). Not a lower bound; half-gcd would give tens. Only Ω(d) kept as the general statement.
4. **D3 C/r is a sampled hit rate.** Marked in the method text and table header. "7.9x" replaced by 7.74–7.91x.
5. **Weight-0 setup artifacts.** Flagged as accounting, not results (0.13, 1.4, 2.1, 0.44). The setup was not re-charged and the table is unchanged, so this erratum's "charge the same setup" fix is open.
6. **5.9x vs 6.15x.** Stated references each time; same-reference comparison 10.3x vs 6.15x of model rho.
7. **D3 / P4 cost equality.** Stated as by construction; only the hit rate was tested.
8. **Prose tags.** Result sentences tagged `[pcc:unverified]`; `lint-prose` run.
9. **Report.** Figure `figures/oracle_search.svg` embedded; `REPORT.pdf` built from this README.

Files

`PROTOCOL.md`, `validator-report.json`, `REPORT.pdf`, `REPORT.typ`. Scripts: `scripts/fp.py` (counted arithmetic, curve), `d1_m2_rootfind.py`, `d2_coset.py`, `d3_torsion_closed.py`, `d5_m3.py`, `d7_lattice.py`, `tradeoff.py`, `model.py`, `make_figure.py`. Data: `data/d1_m2.json`, `d2_coset.json`, `d3_torsion.json`, `d5_m3.json`, `d6_tradeoff.json`,

`d7_lattice.json`, `table.json`. Figure: `figures/oracle_search.{svg,png}`. Reproduce: run each `scripts/d*.py`, then `tradeoff.py`, `model.py` and `make_figure.py`, from this directory (about one minute in total).